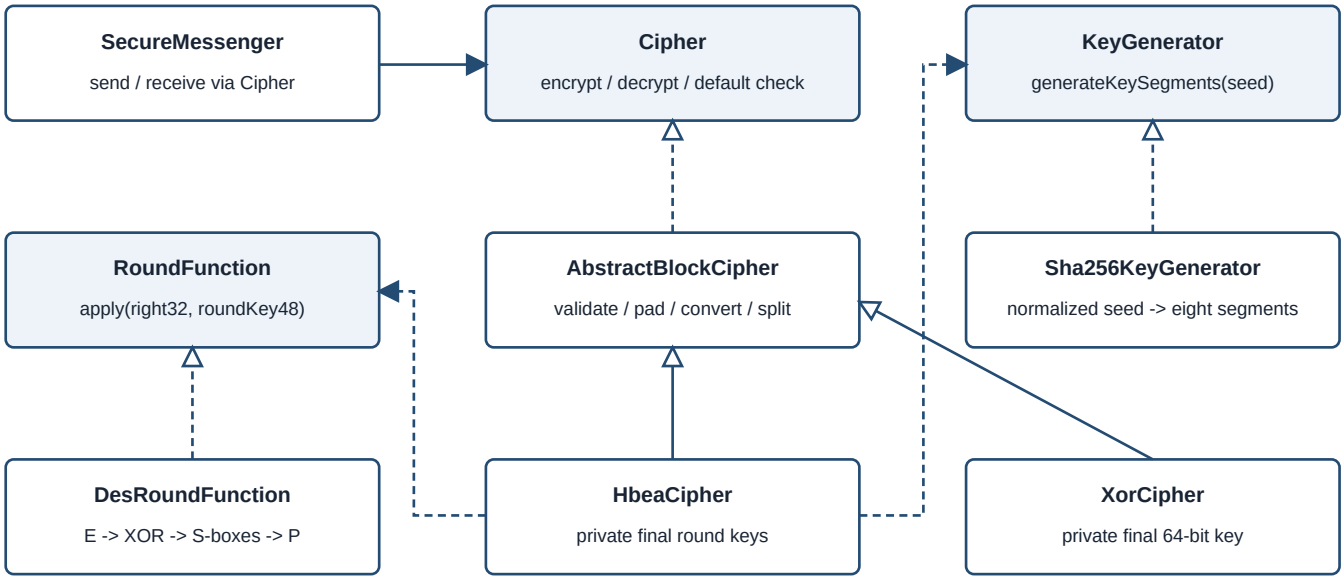
# HBEA Interfaces Project

Matteo Giorgetti | mgiorgetti.ieu2024@student.ie.edu | Java 21

## 1. Design and responsibilities

The program encrypts one message with two different ciphers. Both follow the same **Cipher** interface, so **SecureMessenger** can send and receive messages without knowing which algorithm is being used. The diagram shows the main classes and their dependencies.




Hollow triangle: inheritance or implementation. Filled arrow: uses an interface.

Helpers: BitUtils, HbeaTables, InvalidMessageException. Main creates and runs Cipher objects.

**Shared template.** AbstractBlockCipher implements Cipher. It validates plaintext, adds padding, splits the message into four blocks and converts between bits and hexadecimal. Its abstract encryptBlock and decryptBlock methods leave the block algorithm to each subclass.

**Interfaces and composition.** HbeaCipher receives a KeyGenerator and a RoundFunction. Sha256KeyGenerator and DesRoundFunction provide the required implementations. XorCipher inherits the same input/output handling and XORs each block with the first 64 key bits.

**OOP details.** Cipher includes the default verifyRoundTrip method. KeyGenerator declares SEGMENTS and SEGMENT_BITS. Overloading is shown by encrypt(String) and encrypt(int[]). Overriding methods use @Override. Key fields are private final and have no public getters; DES table arrays are also private. Main stores the ciphers in a Cipher array.

# Algorithm and reflections

## 2. Processing the key and message

**Key generation.** Trim the email and convert it to lowercase using Locale.ROOT. Hash its UTF-8 bytes with SHA-256. Divide the resulting 256 bits into eight consecutive 32-bit segments. Apply DES expansion E to each segment to obtain eight 48-bit round keys.

**Message processing.** Reject null, empty or overlong input and characters outside ASCII 32-126 with the checked InvalidMessageException. Pad valid messages with NUL bytes to exactly 32 bytes. Convert bytes to bits, most significant bit first, and form four 64-bit blocks.

## 3. Encryption and decryption

For each block, apply IP and split it into L0 and R0. Perform eight Feistel rounds with the equations below. The round function expands R to 48 bits, XORs it with the round key, uses eight S-boxes to return to 32 bits, then applies P. In each S-box, the outer two bits select the row and the middle four select the column.

```
L(i) = R(i-1)
R(i) = L(i-1) XOR F(R(i-1), RK(i))
F(R, RK) = P(S(E(R) XOR RK))
```

After round 8, join R8 followed by L8 and apply the final permutation. Join the four cipher blocks as 64 uppercase hex digits. Decryption uses the same block procedure with round keys in reverse order. Convert the recovered bits to bytes and remove trailing NUL padding; ordinary trailing spaces are preserved.

## 4. Reflection questions

**1. Why can decryption work if S-boxes cannot be reversed?** The Feistel network does not need the inverse of F. From L(i) and R(i), the old right half is L(i), and the old left half is R(i) XOR F(L(i), RK(i)). XOR with the same value twice cancels it. Reversing the key order and keeping the final swap therefore recovers the original block.

**2. Why are the last three blocks for "Hi Anas" identical?** The seven-character message and one padding byte occupy the first block. Each remaining block contains only zero bytes. Equal blocks encrypted independently with the same key give equal ciphertext blocks, revealing repetition and patterns in the message.

**3. Why is a key derived only from an email weak?** An email is often public or easy to guess. Anyone who knows it can compute the same SHA-256 key. Hashing a public value does not make it a secret. HBEA is a learning exercise, not a safe messaging system.

**4. What changes for AES?** Add an AesCipher implementing Cipher and use it in the new expression that selects the cipher. Cipher method signatures and SecureMessenger can stay unchanged. A real AES-GCM implementation should handle its own block size, nonce and authentication tag instead of inheriting this fixed 64-bit block template. The format would need to carry the nonce and tag, so the HBEA-only 64-hex-digit rule would not apply to that implementation.

# Execution screenshots

Appendix C | Actual macOS Terminal run using the official university email

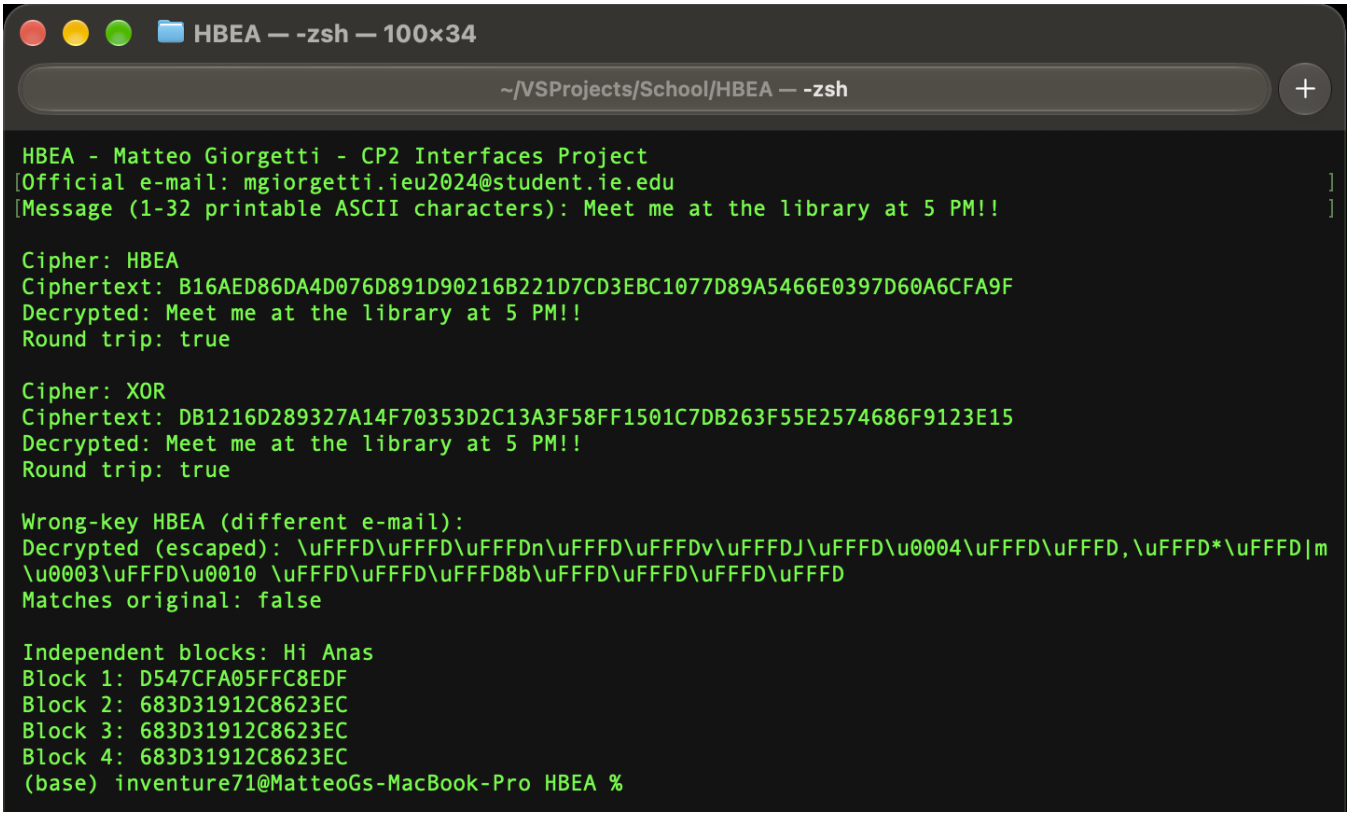



Figure 1. Email and message input, both ciphers, recovered plaintext, wrong-key output and repeated blocks.

## 5. Results from the personalized demo

Both HBEA and XOR report **Round trip: true** for "Meet me at the library at 5 PM!!". The same SecureMessenger class runs both ciphers. The HBEA ciphertext for the official email is:

```
B16AED86DA4D076D891D90216B221D7CD
3EBC1077D89A5466E0397D60A6CFA9F
```

Using a different email produces unreadable text and **Matches original: false**. Non-printable output is escaped in the console so that it cannot alter the terminal display. The last three ciphertext blocks for "Hi Anas" are all **683D31912C8623EC**, which demonstrates the independent-block weakness discussed in reflection 2.

# Testing and round trace

**TDD evidence.** Validation, padding and XOR tests were run against unimplemented methods first: **0 passed, 15 failed** (evidence/tdd-red.txt). After implementation, the corresponding tests pass. Additional BDD scenarios and unit tests check all published vectors, key normalization, wrong keys, polymorphism and byte order.

**Final run: 69 passed, 0 failed.** The checks cover validation/XOR (15), key generation (13), round vectors (12), cipher scenarios (23) and utilities (6). Within the cipher scenarios, each cipher also completes 100 deterministic varied-message round trips. Compilation uses javac -Xlint:all -Werror. Full output is in evidence/tests.txt.

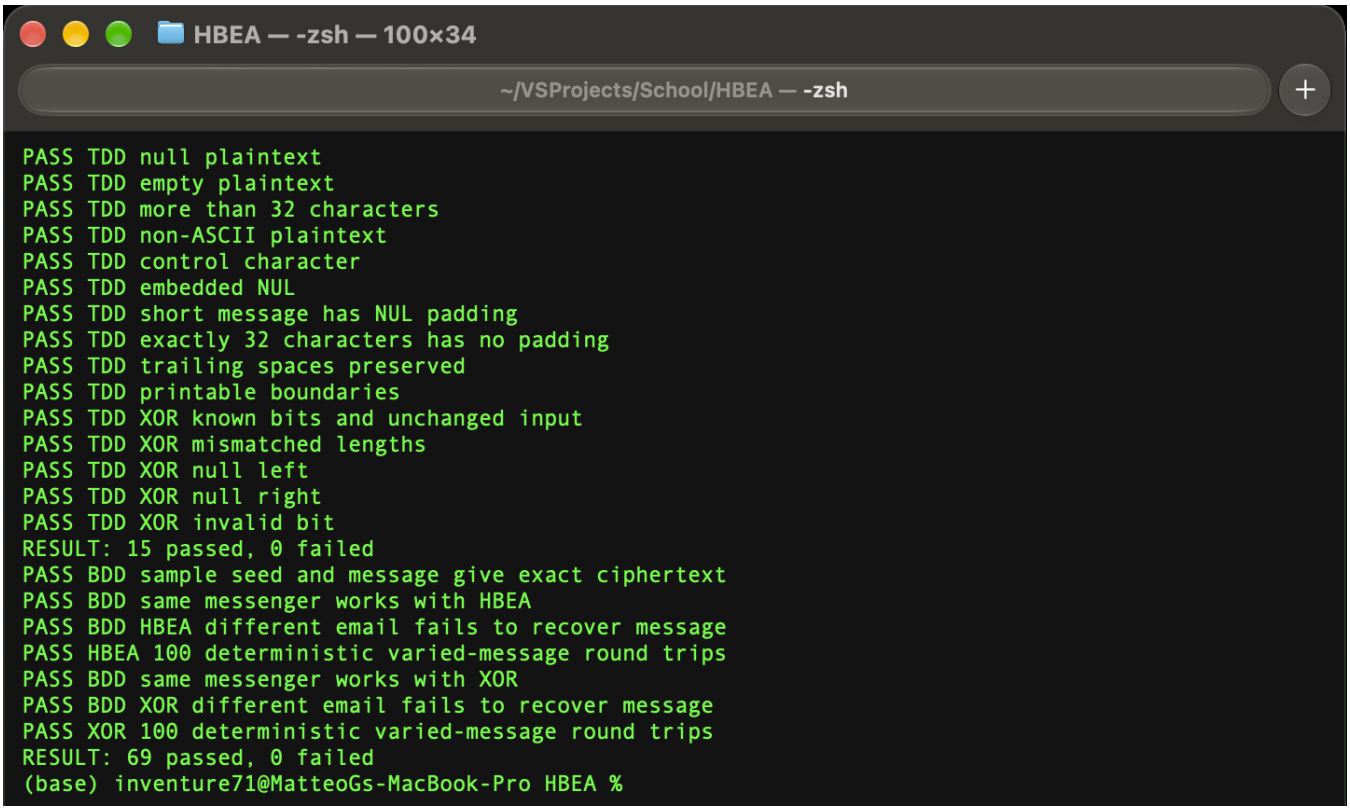



Figure 2. All 15 initial behavior checks pass; selected BDD results and the full-suite total are shown.

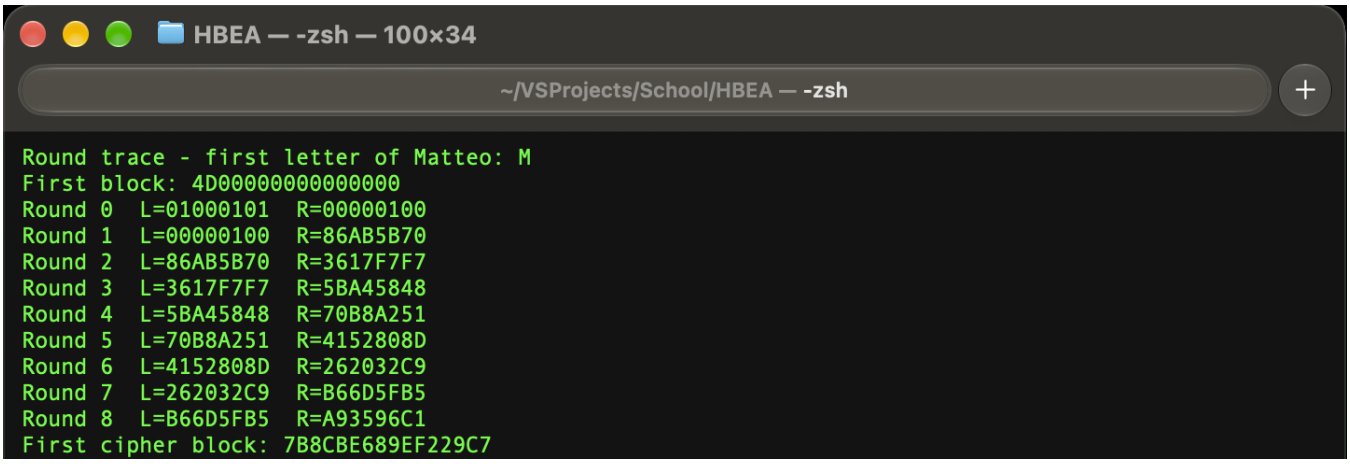



Figure 3. First-letter demo: L and R after every round for M, using the official email. The first encrypted block is 7B8CBE689EF229C7.

**Reproduce:** sh scripts/test.sh; java -cp build Main; java -cp build RoundTrace mgiorgetti.ieu2024@student.ie.edu. The separate DES worksheet is supplied as DES_Worksheet_Solution.pdf and DesWorksheet.java.